Plantillas básicas para copiar

> Archivo personal (no se sube). Cada bloque: **cuándo se usa** + el código para copiar. Cambiás solo lo que dice `x`, `Cosa`, `nombre`.

Front (Angular)

1. Pedir un service (inject)

Cuándo: la pantalla tiene que hablar con el back.

```
private readonly reservas = inject(ReservaService);
```

`inject` se importa de `@angular/core`. Después: `this.reservas.metodo()`.

2. signal: guardar un valor

Cuándo: algo que cambia y se muestra (lo que escriben, si está abierto, un mensaje).

```
protected readonly texto = signal('');        // texto
protected readonly abierto = signal(false);   // sí/no
protected readonly cantidad = signal(0);      // número
```

Hacer	Código
leer	`this.texto()` (en el html: `texto()`)
cambiar	`this.texto.set('hola')`
cambiar según el anterior	`this.cantidad.update((v) => v + 1)`

3. computed: un valor calculado

Cuándo: algo que sale de otros signals (filtrar, contar, sumar). Se recalcula solo.

```
protected readonly filtrados = computed(() =>
  this.recurso.value().filter((x) => x.estado === this.estado()));

protected readonly total = computed(() => this.recurso.value().length);
```

4. rxResource: cargar datos al abrir la pantalla

Cuándo: la pantalla tiene que mostrar algo del back apenas se abre.

```
// sin parámetros
protected readonly recurso = rxResource({
  stream: () => this.servicio.listar(),
  defaultValue: [],
});

// con parámetro (si cambia, vuelve a pedir)
protected readonly recurso = rxResource({
  params: () => this.id(),
  stream: ({ params }) => this.servicio.obtener(params),
});
```

Hacer	Código
el dato	`recurso.value()`
¿está cargando?	`recurso.isLoading()`
¿hubo error?	`recurso.error()`
volver a pedir	`this.recurso.reload()`

`rxResource` se importa de `@angular/core/rxjs-interop`.

5. subscribe: hacer algo cuando tocan un botón

Cuándo: una acción (guardar, borrar, marcar). **El pedido no sale hasta el `subscribe`.**

```
// corto: solo el caso bien
protected marcar(id: Id): void {
  this.servicio.marcar(id).subscribe(() => this.recurso.reload());
}

// completo: bien y mal
protected guardar(): void {
  this.servicio.guardar(this.texto()).subscribe({
    next: () => {
      this.aviso.set('Guardado');
      this.recurso.reload();
    },
    error: (e: Error) => this.error.set(e.message),
  });
}
```

6. Service del front: un método por endpoint

Cuándo: cada ruta nueva del back. Va **adentro de la clase** del service.

```
// GET que devuelve algo
obtenerX(id: Id) {
  return this.http.get<{ x: Tipo }>(`${this.ruta}/${id}/x`)
    .pipe(map(({ x }) => x));
}

// POST (crear) con body
crearX(datos: { nombre: string }) {
  return this.http.post(`${this.ruta}`, datos);
}

// PATCH (cambiar / acción) sin body
marcarX(id: Id) {
  return this.http.patch(`${this.ruta}/${id}/x`, {});
}

// DELETE (borrar / sacar)
borrarX(id: Id) {
  return this.http.delete(`${this.ruta}/${id}/x`);
}
```

- `x` en `get<{ x: Tipo }>` = el nombre que pone el controller en `res.json({ x })`.
- La URL va sin `/api` (se agrega solo). `this.ruta` ya es `'/reservas'`, `'/estacionamientos'`...

7. input / output: padre ↔ hijo

Cuándo: un componente chico (tarjeta, fila) recibe datos y avisa que lo tocaron.

```
// HIJO (.ts)
readonly cosa = input.required<Cosa>();   // recibe
readonly borrar = output<Cosa>();         // avisa
```

```
<!-- HIJO (.html) -->
<button type="button" (click)="borrar.emit(cosa())">Borrar</button>

<!-- PADRE (.html) -->
<app-hijo [cosa]="c" (borrar)="borrar($event)" />
```

```
// PADRE (.ts): el que llama al back
protected borrar(c: Cosa): void {
  this.servicio.borrarX(c.id).subscribe(() => this.recurso.reload());
}
```

8. Plantilla HTML

```
<!-- mostrar un valor -->
<p>{{ texto() }}</p>

<!-- mostrar solo si... -->
@if (abierto()) { <p>Está abierto</p> } @else { <p>Cerrado</p> }

<!-- mientras carga / error / datos -->
@if (recurso.isLoading()) {
  <p>Cargando...</p>
} @else if (recurso.error()) {
  <p>Hubo un error</p>
} @else {
  @for (item of recurso.value(); track item.id) {
    <p>{{ item.nombre }}</p>
  } @empty {
    <p>No hay nada</p>
  }
}

<!-- guardar el valor con un nombre corto -->
@if (recurso.value(); as r) { <p>{{ r.total }}</p> }

<!-- if/else en una línea -->
<span>{{ marcado() ? '♥' : '♡' }}</span>

<!-- botón -->
<button type="button" (click)="guardar()">Guardar</button>
```

9. Inputs conectados a un signal

```
<!-- texto -->
<input [value]="texto()" (input)="texto.set($any($event.target).value)" />

<!-- select -->
<select (change)="estado.set($any($event.target).value)">
  <option value="">Todos</option>
  <option value="PENDIENTE">Pendientes</option>
</select>

<!-- checkbox -->
<input type="checkbox" (change)="activo.set($any($event.target).checked)" />

<!-- número -->
<input type="number" (input)="cantidad.set(Number($any($event.target).value))" />
```

10. Formulario reactivo (alta / edición)

Cuándo: la pantalla ya tiene un formulario (vehículo, estacionamiento) y agregás un campo.

```
// definición
protected readonly formulario = this.fb.nonNullable.group({
  nombre: ['', [Validators.required, Validators.maxLength(60)]],
  activo: false,
});
// precargar al editar (todos los campos)
this.formulario.setValue({ nombre: x.nombre, activo: x.activo });
// leer al guardar
const v = this.formulario.getRawValue();
```

```
<input formControlName="nombre" />
<input type="checkbox" formControlName="activo" />
```

Back (Express)

11. Ruta

```
router.get('/:id/x', ...soloPropietario, controller.verX);
router.post('/', ...soloConductor, validate(validarX), controller.crearX);
router.patch('/:id/x', ...soloConductor, controller.marcarX);
router.delete('/:id/x', ...soloConductor, controller.sacarX);
```

Si no existe el atajo: `const soloConductor = [authenticate, requireRole(ROLES.CONDUCTOR)];`

12. Controller

```
// devuelve algo
export const verX = asyncHandler(async (req, res) => {
  const filas = await servicio.verX(req.params.id, req.usuario.id);
  res.json({ filas });
});

// acción sin respuesta
export const marcarX = asyncHandler(async (req, res) => {
  await servicio.marcarX(req.params.id, req.usuario.id);
  res.json({ ok: true });
});

// crea algo
export const crearX = asyncHandler(async (req, res) => {
  const x = await servicio.crearX(req.usuario.id, req.body);
  res.status(201).json({ x });
});
```

`req.params.id` = el de la URL · `req.usuario.id` = quién pide · `req.body` = el formulario.

13. Service

```
// leer varias filas
export async function listarX(id) {
  const { rows } = await query('SELECT * FROM tabla WHERE id_otro = $1', [id]);
  return rows;
}

// leer una (404 si no está)
export async function obtenerX(id) {
  const { rows } = await query('SELECT * FROM tabla WHERE id_tabla = $1', [id]);
  if (!rows[0]) throw ApiError.notFound('No existe');
  return rows[0];
}

// cambiar
export async function marcarX(id) {
  await query('UPDATE tabla SET x = TRUE WHERE id_tabla = $1', [id]);
}

// crear
export async function crearX(idUsuario, datos) {
  const { rows } = await query(
    'INSERT INTO tabla (id_usuario, nombre) VALUES ($1, $2) RETURNING *',
    [idUsuario, datos.nombre]);
  return rows[0];
}
```

Errores: `ApiError.notFound` 404 · `forbidden` 403 · `conflict` 409 · `badRequest` 400.

14. Validator

```
export function validarX(body) {
  return campos(body)
    .texto('nombre', body.nombre, { max: 60 })
    .texto('nota', body.nota, { requerido: false, max: 200 })
    .entero('cantidad', body.cantidad, { min: 1 })
    .booleano('activo', body.activo, { requerido: false })
    .resultado();
}
```

15. Test

```
describe('x', () => {
  test('hace x', async () => {
    const usuario = await crearUsuario();
    const r = await api('PATCH', `/ruta/${id}/x`,
      { token: usuario.token, body: { campo: 'valor' } });
    console.log(r.estado, r.datos);
    assert.equal(r.estado, 200);
  });
});
```

16. Base (al final de schema.sql)

```
ALTER TABLE tabla ADD COLUMN IF NOT EXISTS nota VARCHAR(200);
ALTER TABLE tabla ADD COLUMN IF NOT EXISTS activo BOOLEAN NOT NULL DEFAULT FALSE;

CREATE TABLE IF NOT EXISTS nueva (
  id_usuario UUID NOT NULL REFERENCES usuario (id_usuario) ON DELETE CASCADE,
  id_estacionamiento UUID NOT NULL
    REFERENCES estacionamiento (id_estacionamiento) ON DELETE CASCADE,
  PRIMARY KEY (id_usuario, id_estacionamiento)
);
```